Risk Alert Service

Monthly Account Risk Alerts → Slack Integration

Daniel Lee

The Problem

What the service has to solve, and why each piece matters

THE SCENARIO

“At Risk” Account Alerting

We need timely, reliable visibility into accounts that are deteriorating so we can intervene before churn. Today, account health lives in cloud storage, containing multiple months of history across every account.

However, there’s no automated mechanism to surface “At Risk” accounts to the right regional team or reliably track how long an account has been struggling.

Data Pipeline

Storage abstraction over gs://, s3://, file://. Single narrowed pass to dedup, filter, and compute.

Business Logic

Walk consecutive months backward, stop on change or gap.

Replay & Resilience

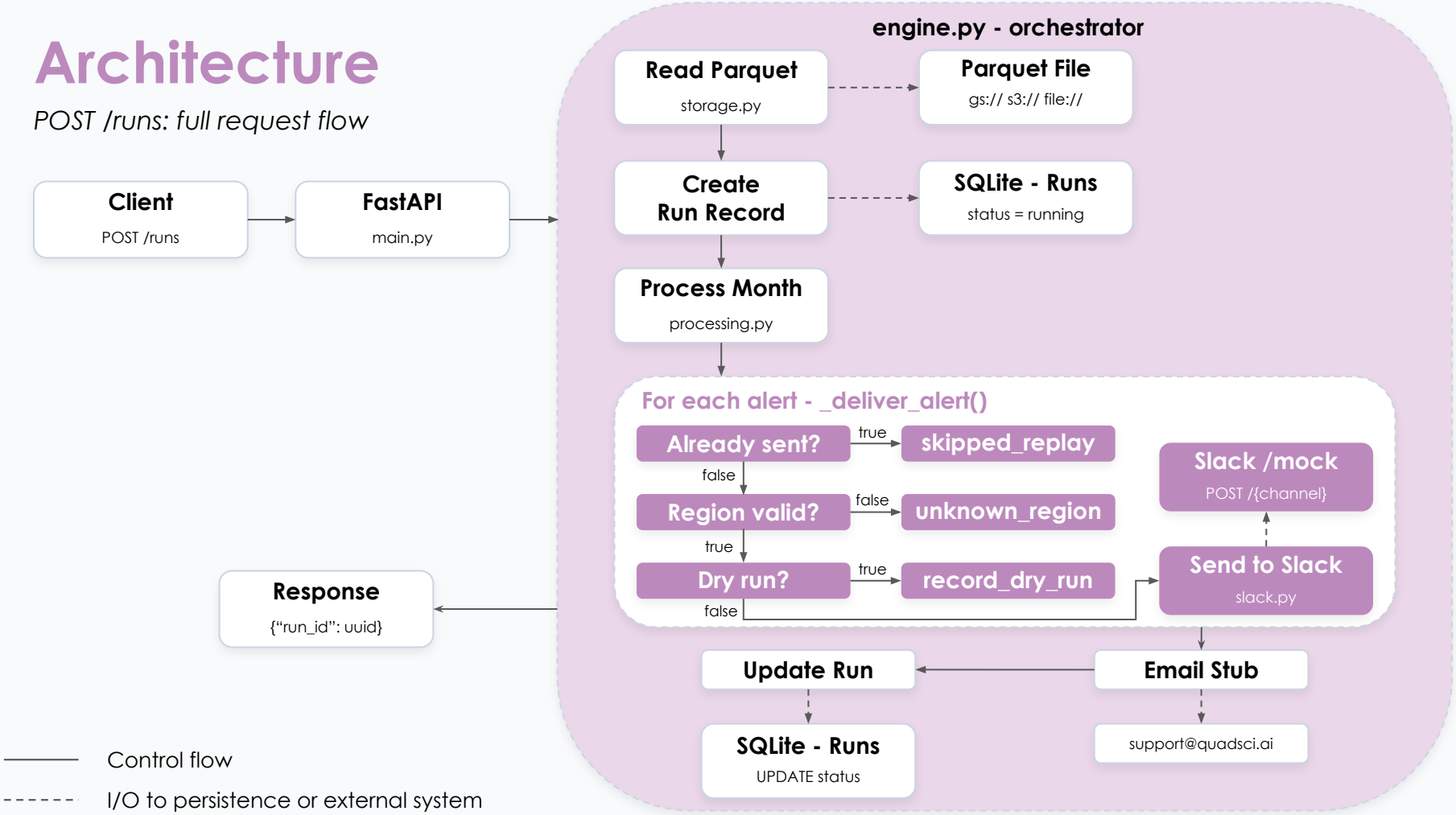
Idempotent logic ensures no duplicate alerts. Failed and dry runs can be safely retried.

Slack Delivery

Retry Slack failures with backoff. No default channel, unknown regions need explicit handling.

Architecture

POST /runs: full request flow



Data Pipeline

Minimize scanning, memory, and I/O

01

Open & Filter

A storage abstraction handles gs://, s3://, and file:// uniformly. PyArrow predicate pushdown and column projection keep reads narrow - only the columns we use, and only months through the target.

02

Deduplicate

Multiple rows can exist for the same account in the same month. Resolve to the most recently written row, and track how many duplicates were dropped so the run summary can surface the count in its response.

03

Compute Risk

Filter to "At Risk" accounts in the target month above the configured ARR threshold, then walk backward to measure the streak. Stop as soon as the status changes or a month is missing. Emit the duration, risk start month, and account metadata needed for the alert payload.

04

Filter & Route

Each surviving alert is tagged with its destination Slack channel via region lookup. Alerts with missing or unrecognized regions are diverted to a separate bucket for aggregated reporting rather than silently dropped.

Business Logic

Important Requirement: A gap resets the streak

THE RULE

Consecutive Months Backward

1. Start at target month (At Risk confirmed)
2. Step back one month at a time
3. Continue while status is At Risk
4. Stop on status change OR missing month
5. Minimum duration is 1 (target itself)
6. Earliest month in the streak = **risk start month**

Implementation: a dict of **(account_id, month)** → status. Every step back is an O(1) lookup.

EXAMPLE

2025-10 → 2026-01

2025-10	2025-11	2025-12	2026-01
At Risk	At Risk	Healthy	At Risk

▲ **TARGET**

← walk back until streak breaks

Result: duration = 1 month

December breaks the streak, so the count restarts at the target month. Even though the account was "At Risk" in October and November, those months don't carry forward across the Healthy gap.

Replay & Resilience

Re-running the same month must never duplicate alerts

MECHANISM

SQLite Unique Constraint

```
UniqueConstraint(
  account_id, month, alert_type
)
```

Before every send, the engine checks alert_outcomes for this key:

- **sent:** [SKIP], already delivered
- **failed:** [RETRY], update existing row
- **dry_run:** [RETRY], update existing row
- **(no row):** [SEND], insert new row

INTEGRATION TEST

Same run, twice

RUN 1

POST /runs month=2026-01-01

alerts_sent: 137 skipped_replay: 0 failed_deliveries: 4

RUN 2: REPLAY

POST /runs month=2026-01-01

alerts_sent: 0 ***skipped_replay: 137*** failed_deliveries: 4

Slack Delivery

Region routing, exponential backoff, graceful handling for unknown regions

RETRY STRATEGY

Exponential Backoff

RETRY ON:

HTTP 429 : rate limited (honors Retry-After header)
HTTP 5xx : server error (transient)

FAIL FAST ON:

HTTP 4xx (non-429): client error, retry won't help

All parameters are env-configurable:
MAX_RETRIES, INITIAL_BACKOFF, MULTIPLIER.

UNKNOWN REGION FLOW

No Default Channel

Unmapped regions must NOT send to Slack, there is no default.

1. **Record as failed**
   status="failed", error="unknown_region"
2. **Aggregate across run**
   Collect all unroutable alerts in memory
3. **Single notification**
   One email to support@quadsci.ai at end of run

Currently written stub, swap in SES or SendGrid in prod

Run Results

POST /runs against the full dataset (January 2026)

10,587
Rows Scanned

–

308
Duplicates Resolved

–

141
"At Risk" Accounts Identified

137
Routable → Slack

4
Unroutable → Email

*Every at-risk account reached some form of notification, **nothing slipped through**.*

Test Coverage

41 tests passing; unit + integration with mocked Slack

41

TESTS PASSING

0 failures ~10s runtime

Duration logic Consecutive months, status change or missing month breaks streak, year boundary	7 tests
Dedup + ARR filter + nulls Latest written date wins, threshold edge cases, nullable region / owner / renewal	8 tests
Replay safety Skip already-sent, retry previously-failed. Both over the full API	2 tests
Slack retry behavior 429 → 200, 500 → 500 → 200, no retry on non-429 4xx	3 tests
Unknown region + dry-run Null region triggers email, unmapped region fails, dry-run persists without sending	6 tests
API contract /health, /preview, /runs, 404 for missing file, 400 for bad scheme	6 tests

Trade-offs & Next Steps

What I chose to defer, and how the system grows next

TRADE-OFF

01 **table.to.pandas()**

Materializes the filtered slice at once.

02 **GCS/S3 temp file download**

Uniform path for local and cloud reads.

03 **Email is a file-write stub**

Interface is production-shaped, only the transport is missing.

PRODUCTION PATH

01 **Record-batch streaming**

Unbounded-memory processing for larger, multi-GB files.

02 **pyarrow.fs direct reads**

Row-group-level reads without full blob download.

03 **Real transport (SES/SendGrid)**

Drop-in behind the existing interface; one line change.

Extensions beyond the trade offs: structured logging with run-scoped correlation IDs, and new alert types like ChurnImminent or ContractionRisk, registered against the existing alert_type constraint.